

```
\points_in_invalid[j], point_to_add))
```

```
number_of_points += 1
```

```
return(delaunay)
```

---

*[Listing 7-3](#): A function that takes a set of points and returns a Delaunay triangulation*

The full DT generation function starts by adding the new outside triangle mentioned earlier. It then loops through every point in our collection of points ❶. For every point, it creates a list of invalid triangles: every triangle that's in the DT whose circumcircle includes the point we're currently looking at ❷. It removes those invalid triangles from the DT and creates a collection of points using each point that was in those invalid triangles ❸. Then, using those points, it adds new triangles that follow the rules of Delaunay triangulations ❹. It accomplishes this incrementally, using exactly the code that we have already introduced. Finally, it returns `delaunay`, a list containing the collection of triangles that constitutes our DT.

We can easily call this function to generate a DT for any collection of points. In the following code, we specify a number for `N` and generate `N` random points ( $x$  and  $y$  values). Then, we zip the  $x$  and  $y$  values, put them together into a list, pass them to our `gen_delaunay()` function, and get back a full, valid DT that we store in a variable called `the_delaunay`:

---

```
N=15
import numpy as np
np.random.seed(5201314)
xs = np.random.rand(N)
ys = np.random.rand(N)
points = zip(xs,ys)
listpoints = list(points)
the_delaunay = gen_delaunay(listpoints)
```

---

We'll use `the_delaunay` in the next section to generate a Voronoi diagram.